

AAAC 编译器设计文档

1 概述

AAAC 是一个编译器工具，用于将 SysY2022 的源程序转换为 GNU 汇编。从功能上划分，AAAC 具体可分为以下三个模块：

- 前端：负责词法分析与语法分析，构建抽象语法树，生成中端中间表示
- 中端：基于控制流图构造 SSA 形式，并进行各类优化，生成后端中间表示
- 后端：先进行寄存器分配，再按照栈帧布局生成最终指令，最后输出汇编

2 前端

2.1 总流程图



2.2 词法分析

AAAC 的词法分析由外部依赖 Flexc++ 协助完成，Scanner 继承自动生成的 ScannerBase，负责维护字符偏移和错误信息缓存，并向 Parser 提供统一的 lex() 接口：

```
class Scanner: public ScannerBase {
public:
    explicit Scanner(std::string fname, std::ostream &out = std::cout)
        : ScannerBase(std::cin, out), pos(0), buf(""), fin(fname),
          errmsg(std::make_unique<frontend::err::ErrMsg>()) {
        switchStreams(fin, out);
    }
    int lex();
    std::unique_ptr<frontend::err::ErrMsg> transferErrMsg();
    int getPos() const { return pos; }

private:
    void adjust(bool update);
    int pos;
    std::string buf;
    std::ifstream fin;
    std::unique_ptr<frontend::err::ErrMsg> errmsg;
};
```

aaac.lex 中定义了所有关键字、运算符和数值字面量的规则，并处理两种注释形式及非法字符。

2.3 语法分析

AAAC 的 Parser 继承 Bisonc++ 的自动生成的 ParserBase，内含一个 Scanner 对象并对 parse()、lex() 等函数进行重写，实现语法与词法的衔接，AAAC 的文法规则定义于 aaac.y

2.4 语义分析

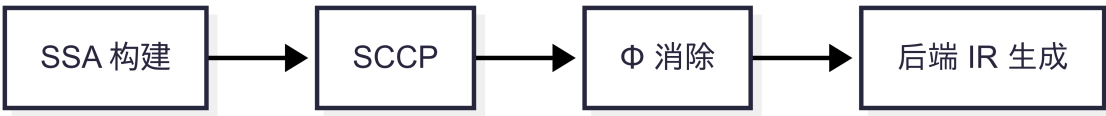
AAAC 的语义分析在 `semant.cc` 中实现，这一阶段主要完成以下任务：

- 1. 列表展开与维度匹配
- 2. 常量属性传播
- 3. 类型推断/匹配
- 4. 变量引用回填与作用域检查

同时，前端会通过 `TUDecl::PopulateLibFunction()` 注入编译期内置函数（参见 `libsusy`），为中端和后端的优化与汇编生成奠定基础。

3 中端

3.1 总流程图



3.2 OpCode 设计

所有运算由 `OpCode` 枚举描述，包括算术、逻辑与类型转换等指令。

```
enum class OpCode : uint16_t {
    Add, Sub, Mul, Div, Mod, Assign, Negate,
    LShift, RShift, BitOr, BitAnd, BitXor, BitNot,
    Integer_cst, Float_cst, AddressOf,
    Eq, Neq, Leq, Lt, Geq, Gt, LogNot
};
```

通用赋值结点 `AssignNode` 记录操作码、目标变量和操作数列表，并实现 `use/def` 查询与替换，便于之后进行数据流分析与重写。

3.3 基本块与控制流图

AAAC 将指令组织为 `BasicBlock` 并通过 CFG 将这些基本块联成图形结构。

`BasicBlock` 保存前驱/后继边、指令列表及可附加的属性类型，`connect/disconnect` 用于维护 CFG 边：

```
void connect(std::shared_ptr<BasicBlock> succ, EdgeFlag flag) {
    this->succs.emplace_back(succ, flag);
    succ->preds.emplace_back(shared_from_this(), PREV);
}
```

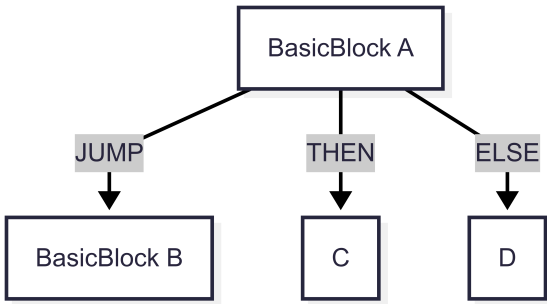


图 1. BB 设计

同时，AAAC 实现了一个 `PropertyManager`，允许为任意对象挂载分析属性（如支配信息、活跃变量等），使用 `type_index` 键值映射管理生命周期，可在多线程环境下安全访问。

3.4 SSA 构建

- 计算支配树
- 计算活跃性
- 插入 ϕ 节点
- 变量重命名

3.4.1 计算支配树

参考LLVM较新的版本(2017年以后)使用的Semi-NCA算法来计算支配树，相较于SLT算法，虽然理论上时间复杂度会高于SLT，但是在LLVM的实践中，Semi-NCA算法要快于SLT算法。同时，这一算法的实现和维护也更为简单一些。

3.4.2 活跃性分析

通过标准数据流方程迭代求解：

$$\begin{aligned} \text{OUT}_B &= \bigcup_{S \in \text{succ}(B)} \text{IN}_S \\ \text{IN}_B &= \text{USE}_B \cup (\text{OUT}_B - \text{DEF}_B) \end{aligned}$$

3.4.3 插入 ϕ 节点

根据变量的所有定义位置 `DefBlocks`，在其迭代支配前沿中插入 `PhiNode`，并记录原始变量以便后续的重命名。在计算迭代支配前沿的时候参考了LLVM中使用的算法，这个算法的时间复杂度近似线性。²

3.4.4 变量重命名

采用标准的变量重命名算法，通过为每一个变量附加一个版本号，整个IR可以拥有SSA性质便于我们进行更多的优化。

3.5 循环分析和规范化

3.5.1 Loop Analysis

Loop Analysis 仅针对自然循环，通过在 Dominator Tree 中寻找回边来识别循环，识别出来的循环会被按照嵌套关系组织成树结构，方便后续对循环的操作。

3.5.2 Loop Simplify

为了简化后续的循环优化，我们对循环进行规范化：

- 添加一个 `preheader` 作为循环外部进入循环头部的唯一入口
- 为所有的 `ExitBlock` 添加前驱
- 保证一个自然循环只有一个回边

3.5.3 Loop-Closed SSA

为了简化循环内部的数据流分析，我们在出口点为每个变量创建一个 PHI 函数，这样可以把循环内外的数据流做一个隔离，便于后续的分析

3.6 优化

3.6.1 SCCP

每个变量对应一个三值格：`UNKNOWN`/`CONSTANT`/`VARIABLE`，工作列表驱动下，指令与 CFG 边交替处理，格合并遵循如下运算：

$$x \sqcap y = \begin{cases} \text{VARIABLE} & \text{if } x = \text{VARIABLE} \vee y = \text{VARIABLE} \\ \text{UNKNOWN} & \text{if } x = \text{UNKNOWN} \vee y = \text{UNKNOWN} \\ \text{CONSTANT} & \text{otherwise} \end{cases}$$

AAAC 的实现中对 ϕ 指令另设 `calculatePhiMeet` 以检查常量一致性，算术指令在两个常量操作数下会直接进行求值，当工作列表为空后，`transform` 阶段将常量替换进 IR，删除死代码并简化不可执行分支

3.6.2 Copy-Propagation

观察到在前端生成的 IR 中会生成许多的临时变量，这些变量一般具有如下的形式：

```
temp = operator result
another variable = temp
```

可以看到，这些临时变量一般只活跃两个指令，但是却引入了不必要的 Copy，因此我们可以通过 Copy-Propagation 消除掉这些临时变量。

3.6.3 LICM

目前的 LICM 算法仅支持对算术运算的不变量外提，算法在一个经过上面的几个循环分析和规范化 Pass 之后的 CFG 上运行。

3.6.4 Simplify CFG

为了删除在 SCCP 和 LICM 等优化 Pass 之后的空基本块，我们使用 Simplify CFG 来进行清理。这个 Pass 会识别出空白的基本块，然后将其删除，同时清理不必要的 PHI 函数等。

3.6.5 Strength Reduction

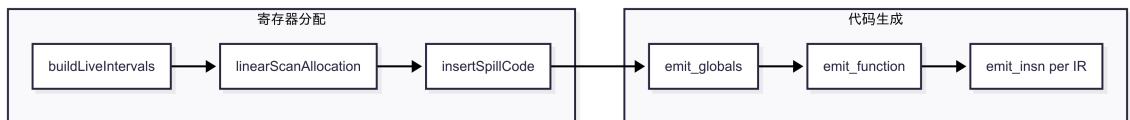
针对某些代价较高的指令（如乘法/除法指令），我们可以把它转化成一个或多个总代价较小的指令。对于有符号除法，在位移前会增加偏置，以符合向零截断的标准。

3.7 后端 IR 生成

`Module::generateInsnList` 将 SSA/CFG 转换为后端指令序列，它会处理全局变量，然后遍历 CFG 生成对应的后端基本块与指令列表。

4 后端

4.1 总流程图



4.2 Peephole

针对中端生成的 `InsnList`，AAAC 在后端提供了一个小范围的窥孔优化 (peephole) 阶段，它会遍历每个函数中的基本块，对指令列表反复应用局部重写规则，直到没有新改动为止。

目前 Peephole 主要的优化规则为:

表格 1.

序列	优化
mv rd, rd	无用语句, 删除
ALU rn, rs1, rs2	
mv rd, rn	ALU rd, rs1, rs2
li rn, 0	
add rd, rs1, rn (或 add rd, rn, rs1)	mv rd, rs1
li rn, 0	
sub rd, rs1, rn	mv rd, rs1
li rn, 0	
sub rd, rn, rs1	negate rd, rs1
li rn, 0	
mul rd, rs1, rn	li rd, 0
li rn, 1	
mul rd, rs1, rn (或 mul rd, rn, rs1)	mv rd, rs1

优化规则都存放在 tryFusion 中, 如果在后续优化中发现重复模式, 可在 tryFusion 中新增, Peephole 就可直接处理这些新增规则。

4.3 寄存器分配

AAAC 采用线性扫描算法进行寄存器分配, 在此过程中, 它通过构建变量的活跃区间、分配寄存器、插入溢出代码、写回分配结果的流程完成整数/浮点寄存器分配, 并与先前的活跃变量分析结果配合使用。

4.3.1 RISC-V 寄存器信息

所有整数/浮点寄存器在 riscv.h 中枚举, 并提供 reg_to_string 与 reg_to_index 接口

```
enum class RISC_V_Reg {
    __zero, __ra, __sp, __gp, __tp,
    __t0, __t1, __t2, __s0, __s1,
    __a0, __a1, __a2, __a3, __a4, __a5, __a6, __a7,
    __s2, __s3, __s4, __s5, __s6, __s7, __s8, __s9, __s10, __s11,
    __t3, __t4, __t5, __t6,
    __ft0, __ft1, :::, __ft11,
    INVALID
};
```

4.3.2 活跃区间建模

LiveInterval 结构记录一个变量的存活区间及分配信息, 支持区间起止、已分配寄存器、是否浮点/溢出及栈偏移等属性:

```
class LiveInterval {
    std::shared_ptr<sym::Var> var;
    int start, end;
    RISC_V_Reg assigned_reg = RISC_V_Reg::INVALID;
    bool Float = false;
    bool spilled = false;
    int spill_offset = -1;
    ...
};
```

通过遍历函数的 BasicBlock 与 CommonInsn 指令, buildLiveIntervals 首先统计每个变量的首次出现位置, 再依据活跃性分析结果延伸区间末端, 最终补入函数调用所需的 caller-save 约束。

4.3.3 线性扫描分配

- **活跃分析**
 - LivenessPass 调用 CFG 提供的 calculateUseDef 与 calculateLiveness, 将基本块级的 in/out 集合转换为后端属性。
- **构建 LiveInterval**
 - buildLiveIntervals 遍历指令列表, 记录每个变量的首次出现位置; 随后结合活跃集信息更新区间的结束点, 从而得到包含起止位置、类型标记等信息的区间列表。
- **线性扫描**
 - linearScanAllocation 按起点排序区间, 维护整数/浮点两个活跃集合与可用寄存器池。预着色变量优先占用指定寄存器, 若无空闲寄存器则选择结束最晚者溢出。
 - 为了减少 Spilled 的变量, 算法末尾还会重新尝试给溢出区间分配寄存器。
- **插入溢出代码**
 - insertSpillCode 统计所需栈空间, 为溢出区间分配栈偏移, 并在相关指令前后插入 Load_sp/Store_sp 指令以完成保存与恢复。

4.4 代码生成

4.4.1 emit_globals

根据 globals.g_registry 中的初始化数据生成 .data/.rodata/.bss 段, 如果是空数组, 则写入 .zero, 同时, 为了减小生成的汇编文件大小, 还使用 zip_init_list 函数压缩零值序列。

4.4.2 emit_function

对函数处理会经历如下步骤:

1. 栈帧计算

- stack_offset 收集函数使用的 callee-saved 寄存器和逃逸变量, 得出对齐后的栈帧大小并为逃逸变量写入 fp_offset。

2. 函数前序

- 根据栈帧大小调整 sp;
- 把 ra/s0 与所有 callee-saved GPR/FPR 压栈;
- 设置 s0 为帧指针。

3. 指令遍历

- 遍历基本块链表, 逐条调用 emit_insn 生成汇编。

4. 函数后序

- 按相反顺序恢复保存的寄存器;
- 还原 sp, 跳转到 ret 标签并执行 ret 指令。

4.4.3 emit_insn

emit_insn 通过 switch-case 语句为不同的 OpCode 生成对应的汇编语句, 在这一过程中, emit_insn 会进行基本的常量折叠与立即数优化, 运算完毕后统一 sext.w rd, rd, 保持 SysY 语义。

同时, 对于浮点转换与混合运算, AAAC 会显式指定 rtz 参数, 确保不受全局 FCSR 影响, 在后端中还提供了 load_to_ft 这一辅助函数来将立即数或整寄存器中的值搬入浮点寄存器。

5 参考资料

1. shecc: A self-hosting and educational C optimizing compiler (<https://github.com/sysprog21/shecc>)
2. Sreedhar and Gao. A linear time algorithm for placing phi-nodes. In Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages. POPL '95. ACM, New York, NY, 62-73.
3. LLVM loop terminology (<https://llvm.org/docs/LoopTerminology.html>)
4. Rice University COMP512 (<https://www.clear.rice.edu/comp512/Lectures/>)